

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA

ARTHUR ERPEN
CAETANO PERUZZO
GABRIEL HERRERA

Relatório sobre a Etapa 2 de Sistemas Operacionais II

Florianópolis
2025

Resumo

Este relatório descreve a implementação da Etapa 2 do projeto de Sistemas Operacionais II: a generalização da API de comunicação desenvolvida na Etapa 1 para suportar múltiplos *componentes* de um mesmo Sistema Autônomo executando como processos independentes dentro de uma mesma máquina virtual. A comunicação intra-VM é implementada por uma nova **Engine** sobre memória compartilhada System V, no modelo produtor $\times$ consumidor com semáforos. O processo *gateway* passa a ser responsável por criar a infraestrutura de IPC e servir de ponte entre a SHM local e a rede entre VMs, garantindo propagação imediata de mensagens em ambas as direções. O texto apresenta os requisitos, a arquitetura, os detalhes de implementação (layout da região de memória, semáforos, ring buffer, *bootstrap barrier*, prioridade de tempo real e empacotamento como biblioteca estática) e os testes automatizados executados em QEMU RISC-V que validam o comportamento sob carga e medem a latência da engine de SHM.

Sumário

1	Introdução	2
2	Visão geral da arquitetura	2
3	Identificadores: slot, porta e PID	3
4	A região de memória compartilhada	4
4.1	Semáforos System V	4
4.2	Ring buffer	5
5	Bootstrap e ciclo de vida	5
5.1	Criação da infraestrutura	5
5.2	Barreira de bootstrap	6
6	Fluxo de envio e recepção intra-VM	7
6.1	Envio	7
6.2	Recepção	7
7	Gateway e roteamento	8
8	Message::Origin: endereço do remetente	9
9	Mecanismos de propagação imediata	9
9.1	Sinalização na SHM via Semáforos	10
9.2	Recepção Assíncrona via SIGIO e O_ASYNC	10
9.3	Roteamento de "Passagem Direta" no Gateway	10
10	Prioridade de tempo real	10
11	Biblioteca estática e organização do build	11
12	Validação experimental	11
12.1	Teste de estresse (stress)	12
12.2	Análise de Latência (rtt vs rtt_intra)	12
13	Discussão e limitações	12
14	Conclusão	13

1 Introdução

Na Etapa 1, cada máquina virtual (VM) executava um único processo que falava diretamente com a `RawSocketEngine` baseada em `AF_PACKET`. A Etapa 2 introduz uma mudança estrutural: cada VM passa a hospedar *múltiplos componentes* (sensores, atuadores, ECUs) como **processos independentes**, coordenados por um processo **gateway**. A comunicação entre esses processos precisa ser feita *dentro da mesma VM* (intra-VM), mas sem abrir mão da possibilidade de continuar falando com componentes remotos através da rede (inter-VM) já usada na Etapa 1.

O objetivo é demonstrar que a API definida na Etapa 1 é suficientemente geral para acomodar essa segunda modalidade de transporte sem alterações no código das aplicações. Para isso, implementamos uma nova **Engine** sobre memória compartilhada System V e passamos a parametrizar o **Protocol** por *duas* NICs, uma sempre presente (SHM) e outra opcional (rede). Só o gateway instancia a segunda.

Requisitos da etapa. :

- a comunicação intra-VM deve usar **IPC System V** (`shmget`, `semget`) em modelo produtor $\times$ consumidor;
- a interface API pública (`Communicator`, `Protocol`, `NIC`) deve permanecer idêntica à da Etapa 1;
- o *gateway* é responsável por criar e destruir a região de memória compartilhada, além de rotear mensagens entre SHM e rede;
- mensagens devem ser propagadas **imediatamente** entre os dois domínios, sem filas intermediárias;
- `Message` deve carregar um `Origin` (*endereço do remetente*) para viabilizar respostas na Etapa 3;
- threads críticas devem rodar em prioridade de tempo real (`SCHED_FIFO`);
- o código de `src/` deve ser empacotado como biblioteca **estática** e os testes em `tests/` devem consumi-la como clientes comuns.

2 Visão geral da arquitetura

Cada VM executa agora um gateway e N processos de componentes. Todos compartilham uma única região de memória (`SHM::Region`) criada pelo gateway, através da qual trocam *frames Ethernet completos* — ou seja, o mesmo formato de pacote usado na rede. A Figura 1 ilustra a topologia interna de uma VM.

Duas engines, um Protocol. A mudança fundamental no núcleo é que o `Protocol` passou a ser um template parametrizado por *duas* NICs:

```
template <typename SharedMemoryNIC, typename RawSocketNIC = void>
class Protocol
    : private SharedMemoryNIC::Observer,
    public Concurrent_Observed<Buffer<Ethernet::Frame>, uint16_t>
{
    SharedMemoryNIC _shm_nic;      // sempre presente
    RawSocketNIC * _socket_nic;    // apenas no gateway
    // ...
};
```

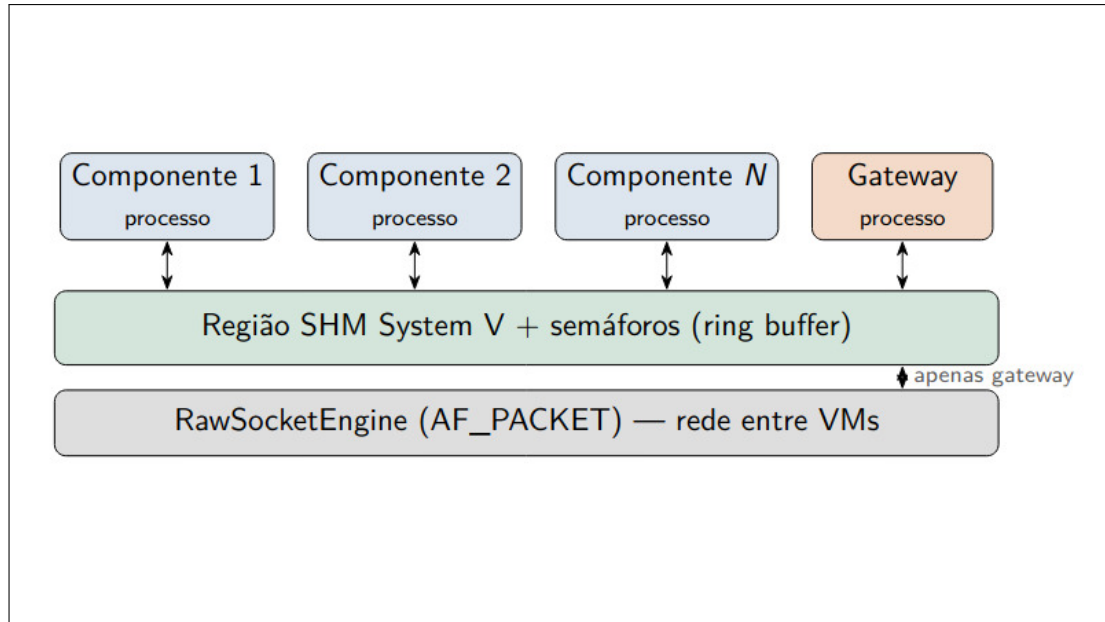


Figura 1: Topologia interna de uma VM na Etapa 2.

A `SharedMemoryNIC` é sempre construída; a `RawSocketNIC` só é alocada quando `SharedMemoryNIC` retorna verdadeiro para `is_gateway_process()`. Dessa forma, componentes comuns nem sequer tentam abrir um socket de pacote bruto — operação privilegiada que, além disso, não faz sentido para eles. A diferença entre gateway e componente é inteiramente resolvida em tempo de execução através do *slot* do processo na região de memória.

3 Identificadores: slot, porta e PID

O `Protocol::Address` manteve a forma `{Physical_Address, Port}`, mas precisamos definir quem atribui essas portas dentro da VM. A primeira ideia natural seria usar o PID do processo como `Port`: é barato, único por VM e trivial de obter. Descartamos essa opção porque **uma VM remota não tem como descobrir o PID de um processo de outra VM** — a comunicação inter-VM ficaria quebrada.

A solução adotada separa dois conceitos:

slot Índice fixo do processo dentro da região de memória da VM, atribuído pelo gateway no momento da configuração. O slot 0 é reservado ao próprio gateway. O slot serve para endereçar os semáforos por-processo e para resolver o *self-drop* na recepção (um processo não deve receber o frame que ele mesmo acabou de escrever no ring).

port Porta lógica do componente (seu “tipo”, e.g. LIDAR, ECU, INS), globalmente conhecida e carregada no cabeçalho do `Protocol::Packet`. É essa porta que aparece nos endereços `Communicator` e que permite descobrir o destinatário mesmo entre VMs.

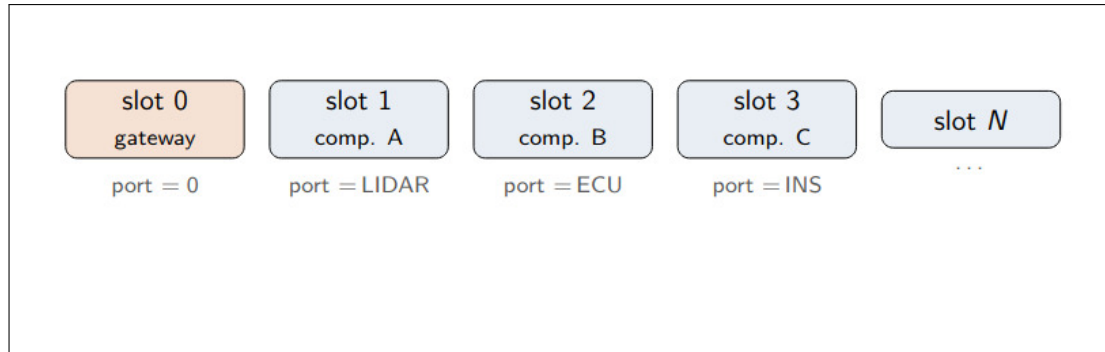


Figura 2: *Slots da shared memory.*

A correspondência (slot, port) é mantida na tabela `Component_Entry` descrita na seção seguinte.

4 A região de memória compartilhada

A região de memória compartilhada (`SHM::Region`) tem o seguinte layout, fixado em tempo de compilação para que gateway e componentes a interpretem exatamente da mesma forma:

```
struct Region {
    uint32_t      magic;                // 0x534F3232 ("S022")
    uint16_t      component_count;
    uint8_t       gateway_active;
    uint8_t       bootstrap_released;
    uint16_t      bootstrap_ready_count;
    Component_Entry components[MAX_COMPONENTS];
    Broadcast_Ring ring;
};
```

magic assinatura "S022" usada para sanity-check no `attach`. Se um processo encontra um segmento antigo com outra assinatura, aborta o startup.

component_count, **gateway_active** contagem de componentes registrados e *flag* que indica se o gateway já completou a criação — usado nos primeiros milissegundos de vida da VM.

bootstrap estado da *bootstrap barrier* descrita na Seção 5.

`Component_Entry[]` tabela (port, slot, active, receiver_ready) indexada pelo slot do processo.

Broadcast_Ring cabeçalho do ring buffer, com `next_write_seq` e `slots[SLOT_COUNT]`. Cada `Broadcast_Slot` carrega `seq`, `writer_slot`, `remaining_readers`, `flags` e um frame Ethernet completo.

4.1 Semáforos System V

A sincronização é feita por um único conjunto (`semid`) com índices fixos, declarados como `enum` para que gateway e componentes sempre falem da mesma posição:

```
enum Semaphore_Index {
    SEM_RING_MUTEX           = 0, // mutex do ring
    SEM_FREE_SLOTS           = 1, // slots livres (inicia em SLOT_COUNT)
    SEM_GATEWAY_PENDING      = 2, // mensagens pendentes para o gateway
    SEM_BOOTSTRAP_MUTEX      = 3, // mutex da barreira de startup
    SEM_BOOTSTRAP_RELEASE    = 4, // liberacao simultanea de todos
};
```

```

    SEM_COMPONENT_PENDING_BASE = 5 // por componente ( pending_i )
};

```

`SEM_FREE_SLOTS` inicia em `SLOT_COUNT` (100) e é o semáforo de controle de fluxo clássico do produtor $\times$ consumidor: um escritor só obtém um slot após um `P(FREE_SLOTS)`. `SEM_RING_MUTEX` protege o estado do ring durante escrita e leitura. Cada componente ativo tem *seu próprio* `pendingi`: o escritor sinaliza individualmente cada leitor ativo, o que nos dá *broadcast natural* e *self-drop* coerente (quem escreveu simplesmente não recebe `V()`).

4.2 Ring buffer

O ring buffer reside em `Broadcast_Ring.slots[]` e tem `SLOT_COUNT=100` posições. A operação típica é:

- **Escritor (writer):** $P(\text{FREE_SLOTS}) \rightarrow P(\text{RING_MUTEX}) \rightarrow$ escreve o slot $\rightarrow$ seta `remaining_readers` com o número de leitores ativos $\rightarrow V(\text{pending}_i) \forall$ leitor ativo $\rightarrow V(\text{RING_MUTEX})$.
- **Leitor (reader):** $P(\text{pending}_{\text{self}}) \rightarrow P(\text{RING_MUTEX}) \rightarrow$ copia `slots[_next_seq % N]` $\rightarrow$ decrementa `remaining_readers`; se zerou, faz `V(FREE_SLOTS)` para devolver o slot $\rightarrow V(\text{RING_MUTEX})$.

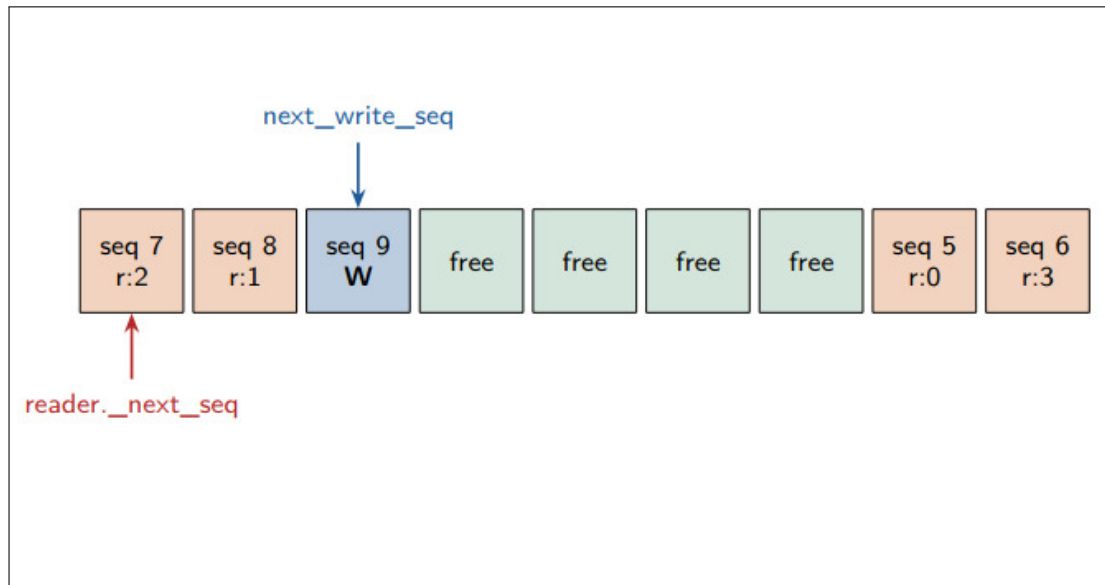


Figura 3: Ring buffer.

As `flags` do slot decidem o roteamento local: `DELIVER_TO_COMPONENTS`, `DELIVER_TO_GATEWAY` e `FROM_NETWORK` permitem distinguir, por exemplo, um broadcast originado pelo próprio gateway (que deve ir para todos os componentes mas *não* ser reenviado para a rede, sob pena de loop) de um frame recém-recebido da rede (que deve ser entregue aos componentes locais mas *não* ao gateway). O campo `writer_slot` permite que cada leitor identifique e ignore sua própria escrita sem depender de informação adicional no cabeçalho do pacote.

5 Bootstrap e ciclo de vida

5.1 Criação da infraestrutura

O gateway é o **único processo responsável** pela criação e pela destruição da região de memória compartilhada. O fluxo completo, implementado em `Vehicle::run` e `Vehicle::run_gateway_process`

(src/application/vehicle.cpp), é:

1. `Vehicle::run()` faz `fork()` de um processo gateway e aguarda nele com `waitpid`.
2. No processo gateway, `run_gateway_process()` chama `SharedMemoryEngine::create()` (que internamente faz `shmget`, `semget`, inicializa os semáforos e o layout da região) e obtém um `Context` com `shmid` e `semid`.
3. O gateway então faz `fork()` de cada componente, passando o `Context` por herança. Cada filho executa `Vehicle::run_component_process`, configura seu slot e porta via `SharedMemoryEngine::configure` e monta seu `Protocol + Communicator`.
4. Após `fork`, o gateway executa seu próprio `run()`.
5. Na saída, também é o gateway quem faz `IPC_RMID` via `SharedMemoryEngine::destroy()`. Componentes são filhos: terminam antes, via `waitpid`, e nunca tocam no `IPC_RMID`.

Esse esquema de *pai único* simplifica a limpeza da SHM: em qualquer caminho de erro, basta encerrar o gateway para que a região seja destruída

```
int Vehicle::run_gateway_process() {
    RT_Priority::set_main_thread_priority("gateway-main");

    SharedMemoryEngine::Context context =
        _gateway.create_context(ports.data(), static_cast<unsigned int>(ports.
            size()));

    if (context.shmid < 0 || context.semid < 0) {
        std::cerr << "[Vehicle] nao foi possivel criar a infraestrutura de SHM."
            << std::endl;
        return 1;
    }

    _gateway.set_context(context);

    for (unsigned int i = 0; i < _components.size(); ++i) {
        pid_t pid = spawn_component_process(i, context);
        if (pid > 0) {
            pids.push_back(pid);
        } else {
            break;
        }
    }
}
```

5.2 Barreira de bootstrap

Há uma janela crítica entre o início do processo componente e o momento em que sua *thread de recepção* consegue efetivamente drenar o ring. Se um escritor publica um frame nesse intervalo, o leitor pode simplesmente perdê-lo (o slot seria devolvido a `FREE_SLOTS` pelo próprio mecanismo de `remaining_readers`). Para fechar essa janela, adicionamos uma barreira explícita:

1. cada componente chama `configure()` e define seu slot/porta;
2. monta `Protocol` e `Communicator`;
3. inicia a thread de recepção e marca `Component_Entry::receiver_ready = 1`;

4. chama `SharedMemoryEngine::wait_until_all_processes_ready()`, que usa `SEM_BOOTSTRAP_MUTEX` e `SEM_BOOTSTRAP_RELEASE` para bloquear até que `bootstrap_ready_count` alcance `component_count` (todos os componentes *mais* o gateway);
5. então todos os processos são liberados praticamente ao mesmo tempo e só agora chamam `Component::run()`.

O resultado é que nenhuma mensagem útil pode ser escrita antes que todos os *receivers* estejam efetivamente aptos a drenar o ring.

6 Fluxo de envio e recepção intra-VM

6.1 Envio

O caminho de envio não mudou do ponto de vista da aplicação: `Component` chama `send(Message)` do `communicator`, que delega para `Protocol::send(from, broadcast, data)`. O `Protocol` aloca um buffer na `NIC<SHM>`, monta o frame Ethernet e chama `engine_send()`. A engine de SHM, dentro de `write_slot()`, executa a sequência de operações descrita na Seção 4: `P(FREE_SLOTS)`, `P(RING_MUTEX)`, `memcpy` do frame para o slot corrente, gravação de `writer_slot` e `remaining_readers`, `V(pendingi)` em cada leitor ativo e `V(RING_MUTEX)`. Ao retornar, o controle sobe pela mesma cadeia (`NIC → Protocol → Communicator → aplicação`), sem nenhum bloqueio adicional.

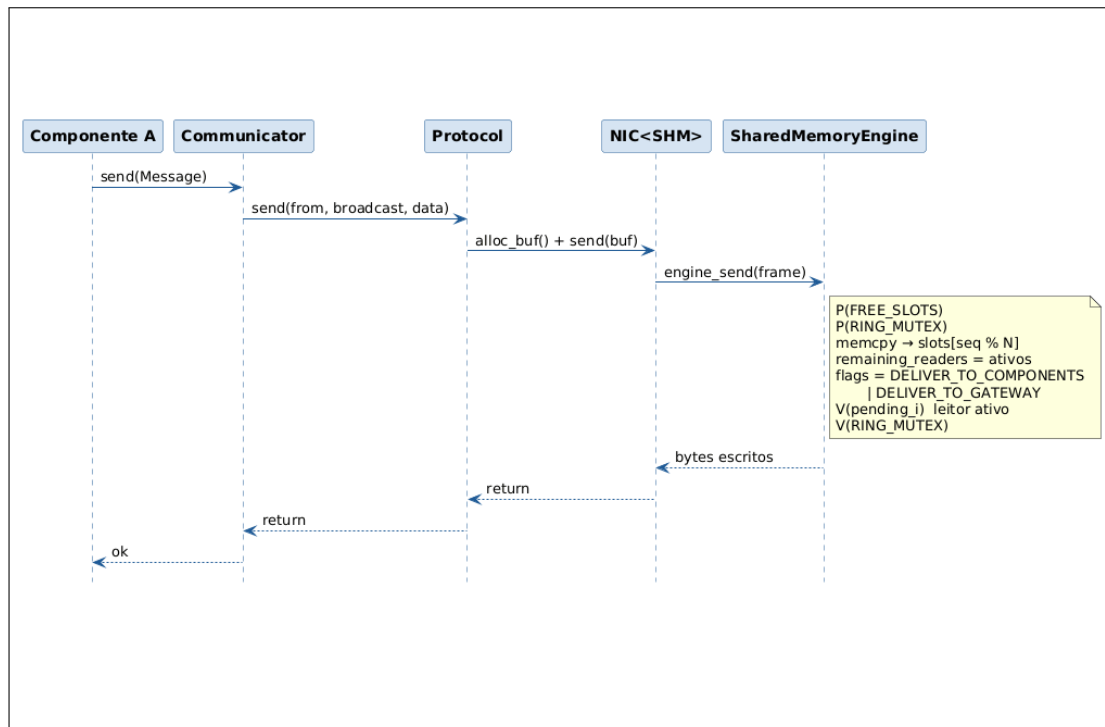


Figura 4: Envio Intra-VM.

6.2 Recepção

A recepção é executada por uma **thread dedicada** por componente, criada durante a construção do `Protocol` e mantida em prioridade de tempo real (Seção 10). O laço da thread bloqueia em `P(pending_self)`; quando retorna, adquire o `RING_MUTEX`, lê o slot corrente (`_next_seq % N`), avalia se deve entregar o frame (*self-drop* via `writer_slot`, máscara de `flags`) e, se for o

caso, reduz `remaining_readers`; quando o contador chega a zero, o slot é devolvido ao pool via `V(FREE_SLOTS)`. O frame é então repassado para a `NIC<SHM>`, que notifica o `Protocol`, que por sua vez faz `dispatch` pelo `dst_port` para o `Communicator` adequado e finalmente desperta a aplicação.

Depois de consumir o primeiro frame, a thread tenta drenar o ring em modo não-bloqueante (`IPC_NOWAIT` nos `semop`) até esvaziá-lo, o que reduz significativamente o número de trocas de contexto sob carga.

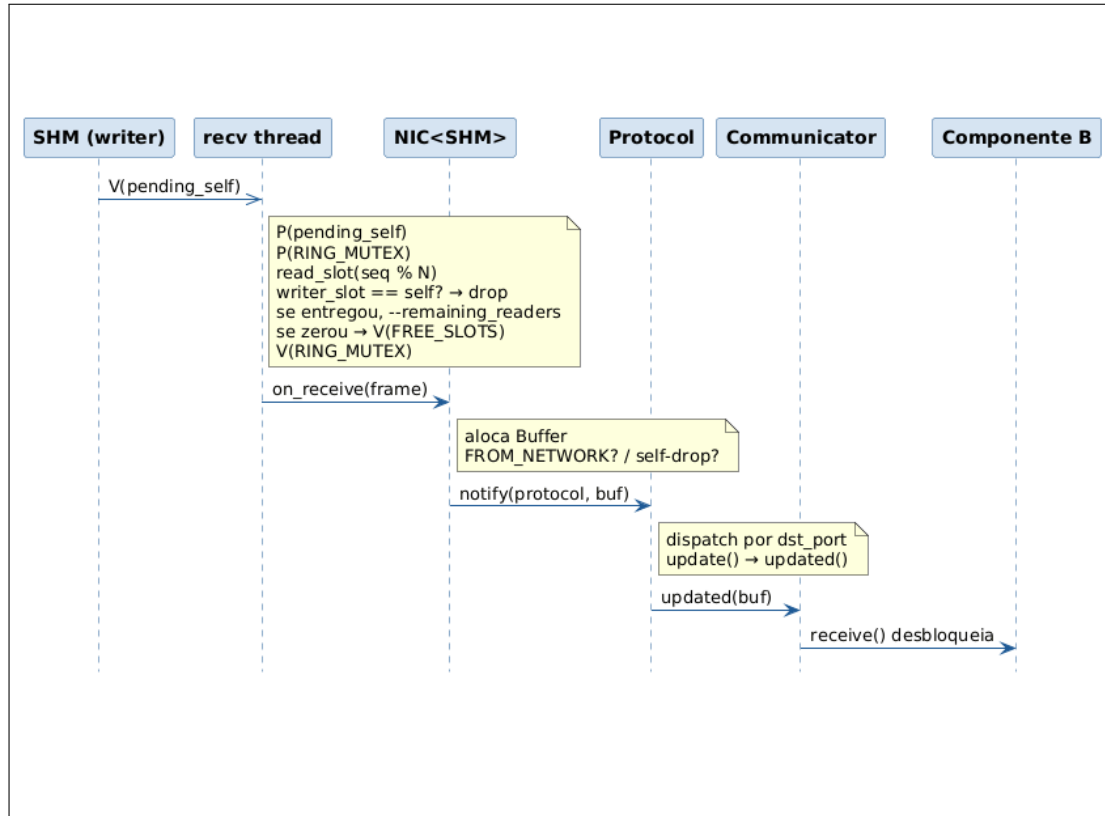


Figura 5: Recepção Intra-VM.

7 Gateway e roteamento

No processo gateway, o mesmo `Protocol` fica *inscrito como observador* nas duas NICs (`SHM` e `Raw`). Isso permite que o método `Protocol::update()` faça o roteamento entre os dois mundos sem nenhuma fila ou thread intermediária:

- **Frame vindo da SHM**, com `src == MAC` do gateway e `dst == BROADCAST`: é um tráfego originado por um componente local que precisa ser visto por outras VMs. O gateway aloca um `Buffer` da `RawSocketNIC`, copia o frame e chama `send` imediatamente — dentro do próprio `update()`.
- **Frame vindo da rede** (entregue pela `RawSocketNIC` via `AF_PACKET`): o gateway aloca um slot na `SHM` e injeta o frame com `flags = DELIVER_TO_COMPONENTS | FROM_NETWORK`. A flag `FROM_NETWORK` garante que o gateway não tentará “reencaminhar” para a rede um frame que acabou de chegar de lá.
- **Qualquer outro caso**: entrega local pelo `dst_port` através dos observadores do `Protocol`, exatamente como na Etapa 1.

```

void update(typename SharedMemoryNIC::Protocol_Number prot, Buffer *buf)
{
    Packet *packet = reinterpret_cast<Packet *>(buf->data()->payload());

    if constexpr (!std::is_void_v<RawSocketNIC>) {
        if (_socket_nic && _shm_nic.owns(buf)) {
            Physical_Address dst_mac = buf->data()->dst();
            Physical_Address src_mac = buf->data()->src();
            if (src_mac == _address._paddr && dst_mac == Ethernet::Address::
                BROADCAST) {
                Buffer* fwd_buf = _socket_nic->alloc(dst_mac, PROTO, buf->
                    size());
                memcpy(fwd_buf->data()->payload(), buf->data()->payload(),
                    buf->size());
                _socket_nic->send(fwd_buf);
            }
        }

        if (_socket_nic && _socket_nic->owns(buf)) {
            Buffer *fwd_buf = _shm_nic.alloc(buf->data()->dst(), PROTO, buf
                ->size());
            std::memcpy(fwd_buf->data(), buf->data(), Ethernet::HEADER_SIZE
                + buf->size());
            _shm_nic.send(fwd_buf);
        }
    }
    bool notified = _observed.notify(packet->dst_port(), buf);

    if (!notified)
        free_buffer(buf);
}

```

8 Message::Origin: endereço do remetente

Para preparar a Etapa 3 (respostas ao remetente), Message passou a carregar um campo Origin:

```

class Message {
public:
    struct Origin {
        Ethernet::Address address; // MAC do remetente
        uint16_t port; // porta logica do remetente
    };
    const Origin & origin() const;
    void origin(const Origin & o);
    // data() / size() ...
};

```

No lado da recepção, Communicator::receive() preenche message.origin({from.paddr(), from.port()}) antes de retornar. O par (MAC, porta lógica) é suficiente para identificar *unicamente* o remetente mesmo que ele esteja em outra VM — o MAC identifica a VM (ou mais precisamente a interface de rede do gateway correspondente) e a porta identifica o componente dentro da VM.

9 Mecanismos de propagação imediata

Um dos requisitos fundamentais da Etapa 2 é que as mensagens sejam propagadas imediatamente entre os domínios de memória compartilhada e rede, evitando latências de *polling* ou filas

de software intermediárias. A implementação atinge esse objetivo através de mecanismos de interrupção e sinalização.

9.1 Sinalização na SHM via Semáforos

Na `SharedMemoryEngine`, a propagação imediata entre processos da mesma VM é garantida pelo uso de semáforos `System V` como canais de interrupção de software.

Diferente de uma abordagem baseada em *busy-waiting* (que consumiria CPU desnecessariamente) ou *polling* temporizado (que introduziria latência fixa), o leitor de SHM bloqueia na chamada de sistema `semop` (operação P) no semáforo `SEM_COMPONENT_PENDING_i`. No momento exato em que o escritor termina de copiar o frame para o ring buffer, ele executa um `V()` nos semáforos de todos os destinatários ativos. O kernel do Linux, então, remove as threads leitoras do estado de espera e as coloca imediatamente na fila de execução, garantindo que o tempo entre a escrita e o início da leitura seja apenas o overhead de escalonamento do sistema operacional.

9.2 Recepção Assíncrona via SIGIO e O_ASYNC

Para a `RawSocketEngine`, a propagação imediata de pacotes vindos da rede física é implementada através de I/O dirigido por sinais (*Signal-driven I/O*).

A engine configura o socket de pacotes brutos com a flag `O_ASYNC` via `fcntl` e define a própria thread de recepção como dona do socket (`F_SETOWN_EX`). Quando um frame chega à interface de rede, o kernel envia um sinal `SIGIO` para a thread. A thread de serviço, que estava bloqueada em um `sigtimedwait`, acorda instantaneamente para drenar o socket. Este mecanismo elimina a necessidade de um laço de leitura bloqueante simples, permitindo que a engine reaja ao evento de hardware (chegada do pacote) com a menor latência possível.

9.3 Roteamento de "Passagem Direta" no Gateway

A arquitetura do `Protocol` no processo gateway consolida a propagação imediata ao eliminar buffers de aplicação entre as engines. O fluxo de roteamento segue uma cadeia de chamadas síncronas:

1. **Evento:** A `RawSocketEngine` recebe um sinal `SIGIO` (ou a `SharedMemoryEngine` é acordada pelo semáforo).
2. **Notificação:** A engine lê o frame e notifica a NIC via callback, que por sua vez invoca `Protocol::update()`.
3. **Decisão e Envio:** Dentro do método `update()`, o gateway identifica se o frame deve ser encaminhado para o outro domínio (ex: SHM → Rede). Se sim, ele chama **imediatamente** o método `send()` da NIC de destino.

Não existem threads de roteamento ou filas `std::queue` intermediárias no gateway. O mesmo contexto de execução que retira a mensagem da SHM é o que a empurra para o socket de rede, garantindo que a latência de trânsito intra-gateway seja virtualmente nula, limitada apenas ao tempo de execução das instruções de cópia e decisão de roteamento.

10 Prioridade de tempo real

As threads críticas do sistema rodam em `SCHED_FIFO` via o helper `RT_Priority`, que aplica o scheduler e ajusta a prioridade, *clampando* para o intervalo disponível retornado por `sched_get_priority_{min,max}`. As prioridades definidas são:

- **main thread** de componente/gateway: prioridade 98, aplicada em `run_component_process` e `run_gateway_process`;
- **thread de recepção SHM**: prioridade 99, o que garante que a recepção não fica atrás de uma main thread “ocupada” processando uma mensagem anterior — se vários frames chegam rapidamente, o scheduler preempta a main a cada `V(pendingi)`.

A API é deliberadamente simples: `RT_Priority::set_main_thread_priority(role)` e `set_service_thread_priority(role)`. Em caso de falha (por exemplo, se o usuário não tem capability `CAP_SYS_NICE`), a função imprime um aviso e segue adiante sem abortar — útil durante o desenvolvimento fora de QEMU.

```
bool set_current_thread_fifo(int priority, const char * role) {
    sched_param param = {};

    const int min_priority = sched_get_priority_min(SCHED_FIFO);
    const int max_priority = sched_get_priority_max(SCHED_FIFO);

    if (priority < min_priority) {
        priority = min_priority;
    } else if (priority > max_priority) {
        priority = max_priority;
    }

    param.sched_priority = priority;
    if (sched_setscheduler(0, SCHED_FIFO, &param) != 0) {
        return false;
    }

    return true;
}
```

11 Biblioteca estática e organização do build

Um requisito explícito da etapa é empacotar `src/` como *biblioteca estática* e reescrever os testes em `tests/` como *clientes* dessa biblioteca. O `makefile` faz exatamente isso:

```
SRCS := $(shell find $(SRC_DIR) -name "*.cpp")
OBJS := $(patsubst $(SRC_DIR)/%.cpp,$(OBJ_DIR)/%.o,$(SRCS))

$(STATIC_LIB): $(OBJS)
    @mkdir -p "$(dir $@)"
    $(AR) rcs $@ $^

$(BIN_DIR)/%: $(TEST_DIR)/%.cpp $(STATIC_LIB)
    @mkdir -p "$(dir $@)"
    $(CXX) $(CXXFLAGS) $< -L$(LIB_DIR) -l$(LIB_NAME) $(LDFLAGS) -o $@
```

Todos os arquivos `.cpp` abaixo de `src/` são compilados em `build/obj/` e colados em `build/lib/libso2.a` via `ar rcs`. Cada teste em `tests/X.cpp` é compilado *separadamente* contra essa biblioteca (`-L build/lib -lso2`), sem conhecer nada sobre a estrutura interna do projeto — ele inclui apenas os cabeçalhos públicos de `src/`. Toda a toolchain usa `riscv64-linux-gnu-g++` com `-static`, o que nos permite rodar os binários resultantes em `qemu-system-riscv64` sem depender de bibliotecas do *rootfs*.

12 Validação experimental

A suíte completa de testes foi executada de forma automatizada em ambiente `qemu-system-riscv64`. O processo de build gera uma biblioteca estática (`libso2.a`) que é vinculada aos binários de

teste, garantindo que o código validado seja idêntico ao que seria utilizado em produção.

12.1 Teste de estresse (**stress**)

O cenário **stress** é o teste mais rigoroso da Etapa 2, pois força a concorrência entre o tráfego local (SHM) e o tráfego de rede (Gateway). Foram utilizadas **5 VMs**, cada uma instanciando um remetente e um receptor. Os parâmetros padrão configurados foram:

- **Volume de dados:** 200 mensagens por VM (total de 1.000 mensagens circulando no sistema);
- **Interstício de envio:** 500 μ s entre mensagens de dados;
- **Sentinelas de finalização:** 200 mensagens DONE enviadas após os dados para garantir o esvaziamento das filas;
- **Janela de startup:** 10 segundos para estabilização do roteamento e barreira de bootstrap.

A suíte foi executada múltiplas vezes em três computadores distintos. Na quase totalidade dos experimentos, o sistema apresentou zero perdas, confirmando a eficácia da sincronização via semáforos. Observou-se, contudo, uma única ocorrência em um dos computadores de teste onde houve a perda de *23 pacotes das 1.000 totais enviadas* (taxa de 2,3%).

12.2 Análise de Latência (**rtt** vs **rtt_intra**)

Para medir o impacto da nova arquitetura de memória compartilhada, comparamos o RTT (*Round-Trip Time*) de mensagens entre componentes em VMs distintas (via rede) contra mensagens entre componentes na mesma VM (via SHM). Os resultados agregados de 20 amostras estão resumidos na Tabela 1.

Tabela 1: Comparativo de Latência Agregada (em milissegundos)

Cenário	Mínimo	Médio	Máximo
Inter-VM (rtt)	3,3470 ms	19,7929 ms	733,1130 ms
Intra-VM (rtt_intra)	1,5670 ms	2,7880 ms	13,3420 ms

Análise dos resultados: A latência média intra-VM (**2,78 ms**) é aproximadamente 7 vezes menor que a latência inter-VM (**19,79 ms**). Isso demonstra que o overhead introduzido pela camada de semáforos e cópia de memória System V é pequeno comparado ao custo de atravessar o stack de rede (AF_PACKET) e a infraestrutura virtual de rede do QEMU.

O valor máximo de **733,11 ms** no teste inter-VM reflete o impacto de preempções do sistema operacional hospedeiro sobre o QEMU sob carga, enquanto a consistência do **rtt_intra** (máximo de apenas 13,34 ms) reforça que a comunicação via memória compartilhada é muito mais resiliente a variações externas, sendo a escolha ideal para componentes críticos de controle local.

13 Discussão e limitações

Algumas decisões do projeto merecem comentário explícito:

Propagação imediata e Signals: Diferente da `RawSocketEngine`, que utiliza sinais assíncronos (`SIGIO`) para reagir à chegada de frames, a `SharedMemoryEngine` baseia sua propagação imediata na sinalização de **semáforos**. O receptor fica bloqueado sincronamente no kernel através do `semop`; no instante em que o escritor realiza o `V()` no semáforo correspondente, o escalonador do kernel acorda o receptor. Os testes de RTT confirmam que esse mecanismo é suficiente para atingir latências na casa dos microssegundos em hardware real, traduzindo-se nos baixos milissegundos observados no ambiente virtualizado.

Frames Ethernet como unidade de transporte intra-VM. A opção de passar *frames completos* pela SHM foi tomada para que o `Protocol` pudesse ser reutilizado *sem modificação*. O custo é uma leve sobrecarga de `memcpy` do cabeçalho Ethernet; o benefício é que todo o trabalho de roteamento no gateway vira uma simples decisão sobre `flags` e endereços, sem tradução de formato.

Semáforos pending, por componente. Com um semáforo por leitor, cada escrita faz exatamente `N V()`s e cada leitor acorda exatamente uma vez por frame dirigido a ele, minimizando a contenção no `RING_MUTEX`.

Limitações conhecidas.

- O `MAX_COMPONENTS` e `SLOT_COUNT` são estáticos (100 cada);
- A recuperação de *crash* de componentes é parcial (o slot é marcado inativo, mas contadores de leitores pendentes em frames já escritos podem demorar a zerar);

14 Conclusão

A Etapa 2 confirma que a `API Communicator / Protocol / NIC / Engine` é suficientemente geral. O gateway assume tanto o papel de criador da infraestrutura de IPC quanto o de *bridge* entre a SHM local e a rede externa. Os testes de estresse com 1.000 mensagens e os benchmarks de latência validam que a solução é robusta e performática, permitindo que componentes locais se comuniquem com latência média inferior a 3 ms, enquanto mantém a transparência para comunicação remota. Com isso, o sistema está pronto para a Etapa 3, que introduzirá a capacidade de resposta dirigida ao remetente.